

Exercise 1 – Box Detection Discussion

1. Goal and Scope of the Individual Task

The individual part extends the original box-detection pipeline developed for the group exercise. The group solution already detected the floor plane, removed it from the point cloud, detected the top plane of the box, extracted the largest connected component, and estimated the physical box dimensions. The individual task focuses on a deeper evaluation of the RANSAC-based plane-fitting step.

The implementation compares three manually implemented plane-fitting strategies:

1. Standard RANSAC
2. MLESAC-style RANSAC
3. Preemptive RANSAC

In addition to the method comparison, the implementation performs hyperparameter tuning, runtime analysis, and a simple paired hypothesis test for the runtime comparison. Therefore, the objective is not only to estimate the box dimensions, but also to understand how the different RANSAC strategies behave with respect to robustness, accuracy, and speed.

2. Pipeline Overview

The individual implementation keeps the same basic processing pipeline as the group solution:

- ✓ Load Kinect .mat file
- ✓ Read amplitude image, distance image, and point cloud
- ✓ Ignore invalid 3D points where $z = 0$
- ✓ Estimate the floor plane using one RANSAC variant
- ✓ Convert floor inliers into a 2D mask
- ✓ Clean the floor mask using morphological closing and opening
- ✓ Remove floor points from the point cloud
- ✓ Estimate the box-top plane using the same RANSAC variant
- ✓ Convert top-plane inliers into a mask
- ✓ Clean the top mask and keep the largest connected component
- ✓ Estimate height, length, and width of the box
- ✓ Save visualizations, CSV files, JSON summaries, and comparison plots

The main difference from the group version is that the plane-fitting step can now be executed with three different RANSAC strategies. This makes it possible to analyse whether the estimated plane, runtime, and quality score change when the model-selection strategy changes.

3. Standard RANSAC Baseline

Standard RANSAC is used as the baseline. It randomly samples three 3D points, estimates a candidate plane, and counts how many evaluation points lie within a predefined distance threshold. The candidate with the largest inlier count is selected as the best model.

This approach is suitable for the box-detection task because both relevant surfaces the floor and the top of the box are approximately planar and contain many inlier points. The method is also robust to outliers, since points from other surfaces do not contribute to the consensus count of the dominant plane.

However, the baseline has two important limitations:

- It uses a binary inlier/outlier decision and does not distinguish between very good inliers and borderline inliers.
- It may spend unnecessary computation on many weak candidates, especially when the scene is already simple or when many candidates are clearly poor.

These limitations motivate the two individual extensions: MLESAC-style scoring and preemptive RANSAC.

4. MLESAC-Style Extension

MLESAC modifies the scoring strategy of standard RANSAC. Instead of only counting how many points fall within the threshold, it evaluates the residual distribution of the points. A point with a very small residual contributes more strongly than a point that is only barely inside the threshold.

The implementation follows a mixture-model idea:

- Inliers are modelled with a Gaussian residual distribution.
- Outliers are modelled with an approximately uniform residual distribution.
- The candidate plane with the best overall likelihood score is selected.

This makes the selection less dependent on a strict threshold boundary. For example, two candidate planes can have similar inlier counts, while one of them fits the inliers much more tightly. Standard RANSAC may treat both models as nearly equal, whereas MLESAC can prefer the model with smaller residuals.

Advantages of MLESAC	Limitations of MLESAC
Uses residual quality, not only inlier count.	More computationally expensive than simple inlier counting.
Can select a cleaner plane when candidate inlier counts are similar.	Requires additional likelihood parameters such as noise scale and inlier prior.
Usually more robust to borderline points near the threshold.	May offer only a small improvement in very clean scenes.

For this task, MLESAC is mainly a robustness improvement. It becomes especially useful when the point cloud contains noisy object boundaries, partial occlusions, or weakly separated planar surfaces.

5. Preemptive RANSAC Extension

Preemptive RANSAC is designed to improve computational efficiency. Instead of evaluating every candidate plane on all points, it first generates a pool of candidate planes and then evaluates them gradually on small blocks of points. After each block, weak candidates are removed, and only stronger candidates survive to later evaluation stages.

The procedure can be summarized as follows:

- Generate many candidate planes
- Evaluate candidates on a small subset of points
- Remove the weakest candidates
- Evaluate the remaining candidates on another subset
- Repeat until only a few strong candidates remain
- Select the final best candidate

This saves computation because clearly poor candidates are not evaluated on the complete point set. The implementation also performs a final scoring step among the surviving candidates to make the final model choice more stable.

Advantages of Preemptive RANSAC	Limitations of Preemptive RANSAC
Can reduce runtime by avoiding full scoring of weak candidates.	A good candidate can be removed too early if pruning is too aggressive.
Useful when many candidate models are generated.	Requires careful settings for candidate count, block size, and minimum survivors.
Provides a practical speed-oriented alternative to standard RANSAC.	Runtime improvement depends on the dataset and implementation overhead.

A key implementation detail is that candidates are pruned only after they have been evaluated on comparable evidence. This is important because premature pruning can incorrectly discard a potentially good model.

6. Hyperparameter Tuning

The individual code performs a compact hyperparameter sweep over the following factors:

- RANSAC inlier threshold
- Maximum iteration budget
- RANSAC variant
- Example index

The default tuning grid is:

thresholds: 0.015, 0.020, 0.025, 0.030, 0.035 m

iterations: 150, 300, 600

The purpose of this tuning experiment is to show how sensitive the final estimates are to the selected parameters. This is important because manually chosen thresholds can work for a small test set without demonstrating whether the method is stable under parameter variation.

The tuning plots visualize changes in estimated length, width, height, runtime, and internal quality score. Since no ground-truth box dimensions are provided with the data, the best setting is selected using an internal consistency score based on:

- parallelism between the floor plane and the box-top plane;
- plausible box height;
- agreement between height estimated from point-to-floor distances and height estimated from plane offsets;
- size of the detected connected component for the box top.

The internal quality score is not a true accuracy metric. It is a practical consistency measure for comparing parameter settings when ground truth is unavailable.

7. Runtime and Speed Analysis

The runtime analysis runs the three RANSAC variants multiple times over the examples and records the total runtime. The generated files allow direct comparison of whether preemptive RANSAC provides a measurable speed advantage in practice.

results_individual/individual_timing_runs.csv

results_individual/individual_timing_summary.json

results_individual/individual_timing_summary.png

This evaluation is useful because theoretical speed improvements do not always appear on small or clean datasets. Preemptive RANSAC can save time by pruning candidates, but it also introduces overhead for

candidate management and staged evaluation. Therefore, the timing experiment provides an empirical basis for the discussion.

8. Runtime Hypothesis Test

A simple paired runtime hypothesis test is included to structure the speed comparison:

H0: Standard RANSAC and preemptive RANSAC have equal mean runtime.

H1: Standard RANSAC is slower than preemptive RANSAC.

The result is saved in:

results_individual/individual_hypothesis_test_runtime.json

This test should be interpreted carefully because the number of examples is small. It is not intended as a final scientific proof, but it provides a transparent and structured way to discuss whether the measured speed difference is meaningful for this exercise.

9. Visualization Outputs

Output type	File pattern	Purpose
Input visualization	example*_inputs.png	Shows amplitude image, distance image, and subsampled 3D point cloud.
Detection visualization	example*_<method>_masks.png	Shows raw/filtered masks and final box-top component.
3D visualization	example*_<method>_3d.png	Shows detected floor, box top, and estimated top outline in 3D.
Presentation visualization	example*_<method>_presentation.png	Shows a clean colored result with floor, box top, outline, and labels.
Method comparison plots	example*_method*_comparison.png	Compares height, runtime, and quality score between methods.
Tuning plots	example*_<method>_tuning_*.png	Shows sensitivity to threshold and iteration settings.

10. Advantages and Disadvantages

Method	Main advantages	Main disadvantages
Standard RANSAC	Simple, robust to outliers, effective when the dominant plane has many inliers, and easy to interpret.	Depends strongly on the inlier threshold; binary scoring ignores residual quality; runtime can increase when the inlier ratio is low.
MLESAC-style RANSAC	Uses residual quality and can prefer tighter plane fits when inlier counts are similar.	More expensive to evaluate and introduces likelihood-related parameters.
Preemptive RANSAC	Designed for speed by pruning weak candidates before full evaluation.	Can fail if pruning is too aggressive or if early evaluation blocks are not representative.

11. Overall Conclusion

The original two-stage RANSAC pipeline is already a solid solution for the provided box-detection examples. The individual extension improves the evaluation by testing whether alternative RANSAC strategies provide better robustness or speed.

MLESAC mainly improves the model-selection criterion. It rewards planes that fit the data tightly and can therefore be more stable when the inlier threshold is difficult to choose. Preemptive RANSAC mainly improves the runtime strategy by spending less computation on weak candidate planes.

The hyperparameter tuning and visualizations are important because they reveal whether the final box dimensions are stable or sensitive to small parameter changes. The runtime analysis and hypothesis test further strengthen the discussion by making the speed comparison measurable rather than purely qualitative.

For the final submission, the most important outputs are:

- the updated Python script;
- the method-comparison CSV and JSON files;
- the hyperparameter tuning plots and best-parameter summary;
- the runtime summary and hypothesis-test JSON file;
- the selected visualizations or compact summary images;
- this discussion document.